DESIGN DOCUMENT

Course: **Operating Systems [CS2012E]**

Assignment: **Assignment 2 -** *Verifying if the given number is perfect with Multithreading*

Submitted by:
Name: **Kona Bhuvaneswar Reddy**
Roll No: **B240214CS**
Class: **CS02**
Date: **10th April, 2026**

Introduction

- **Problem Statement:** The goal is to determine if a user-provided integer ***N*** is a “Perfect Number” by utilizing ***P*** parallel threads. This approach demonstrates how multithreading can decompose a computational task to improve performance.
- **Definitions:** A **Perfect Number** is a positive integer that is equal to the sum of its proper divisors (all positive divisors excluding the number itself). Mathematically, ***N*** is perfect if ∑factors = ***N***.

Workload Partitioning

- **Optimization:** Instead of checking every number from **1** to ***N***, the algorithm only searches up to $\sqrt{N}$ (limit)**.** Factors come in pairs: if ***i*** is a factor, then ***N/i*** is also a factor. This reduces the time complexity from O(***N***) to O($\sqrt{N}$), which is significantly faster for large values of ***N***.
- **Division Logic:** The range [**1**,$\sqrt{N}$] is partitioned into ***P*** segments. Each thread is assigned a `start` and `end` index. The range size is calculated as limit***/P***. To handle cases where the limit isn't perfectly divisible by ***P***, the final thread is assigned all remaining values up to the limit.
- **Example:** For ***N***=10000, $\sqrt{N}$=100. If ***P***=4, the workload is:
  - *Thread 0*: 1–25
  - *Thread 1*: 26–50
  - *Thread 2*: 51–75
  - *Thread 3*: 76–100

Thread Synchronization

- **Shared Resources:** The primary shared resource is the `total_sum` variable, which accumulates the sum of all discovered factors, and the `lock` (mutex), which is used for synchronization to ensure that multiple threads do not access the shared sum simultaneously.
- **The Race Condition:** Without synchronization, a race condition occurs when two threads attempt to read, increment, and write back to `total_sum` simultaneously. One thread's update could overwrite the other's, leading to an incorrect total and a false "**false**" result.
- **Synchronization Primitive:** We use `pthread_mutex_t` (a mutual exclusion lock).
  - **Locking:** `pthread_mutex_lock` ensures that only one thread can access the critical section (updating the sum) at a time.
  - **Unlocking:** `pthread_mutex_unlock` releases the lock, allowing the next waiting thread to enter the critical section.

System Flow (Process Lifecycle)

A step-by-step breakdown of the execution:

1. **Main Process:** Reads ***N*** and ***P*** from the command line, validates ***N***>1, and initializes the thread attributes and mutex.
2. **Creation:** The main loops ***P*** times, calculating ranges and calls `pthread_create` to spawn the threads.
3. **Execution:** Each thread iterates through its assigned range independently. If a factor is found, it adds both the factor and its pair to its `local_sum`. Once the loop finishes, the thread waits in the queue, locks the mutex to add its `local_sum` to `total_sum`, and then unlocks the mutex.
4. **Join (Synchronization):** The main calls `pthread_join` for each thread identifier, acting as a barrier to ensure all calculations are finished by the threads and exited.
5. **Final Check:** The main compares `total_sum` to ***N***. If they are equal, it prints "**true**"; otherwise, it prints "**false**".

Data Structures

- **ThreadData Struct:**

```
typedef struct{
    long long N;
    long long start;
    long long end;
    long long *shared_sum;
    pthread_mutex_t *lock;
} ThreadData;
```

- **Reasoning:** The `pthread_create` function only accepts a single `void*` argument. To pass multiple parameters (the range, the given number, and pointers to the shared sum/lock), we encapsulate them into a single structure.

Testing and Results

Input (*N*)	No of Threads (*P*)	Result	Observation
6	2	**true**	Smallest perfect number (1+2+3)
28	4	**true**	Correctly identifies 28
10	2	**false**	Sum is 8 (1+2+5), correctly fails
8128	8	**true**	Large perfect number verified successfully
1	1	**false**	edge case
28	100	**true**	$P > \sqrt{N}$ handled by range logic.
6	0	**ERROR**	Invalid number of threads

Trade-offs and Optimization

- **Lock Contention:** A naive approach would lock the mutex every time a factor is found. In this design, we use a **Local Sum Accumulator**. By having each thread maintain a `local_sum` and only accessing the global `total_sum` **once** at the end of its execution, we minimize "lock contention". This allows threads to run almost entirely in parallel without waiting for one another, maximizing CPU utilization.

References

- Google.com
- Class Slides